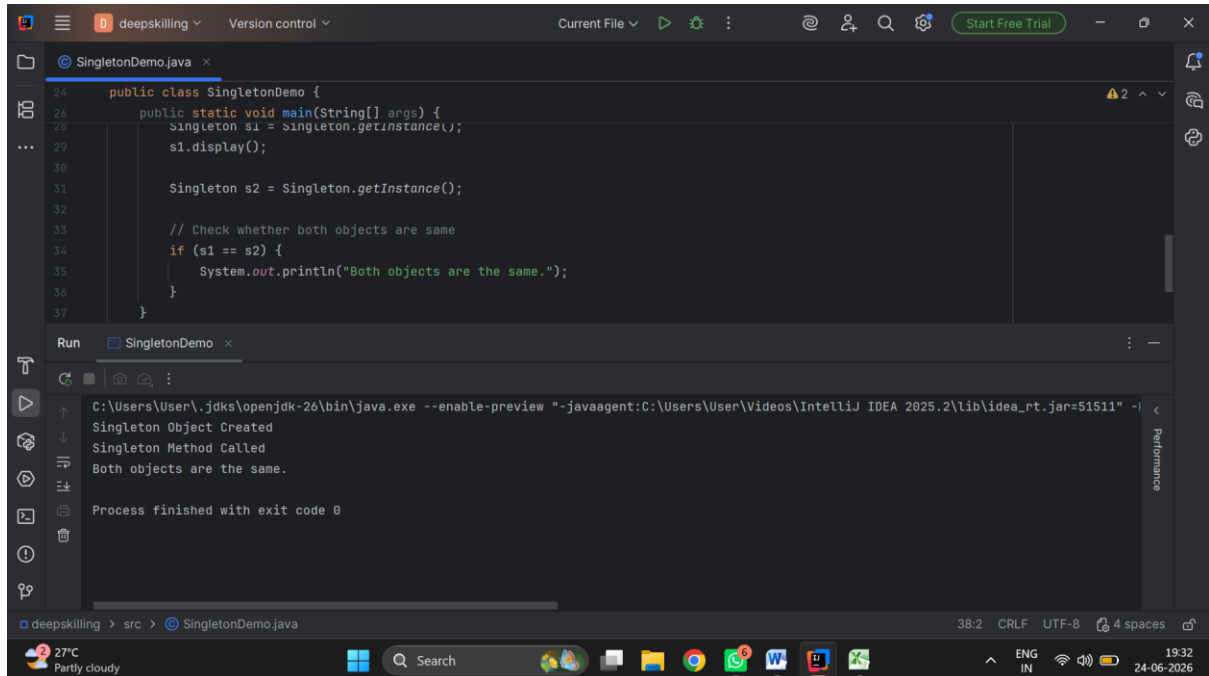


Deepskilling

Design principles & Patterns

Exercise 1: Implementing the Singleton Pattern



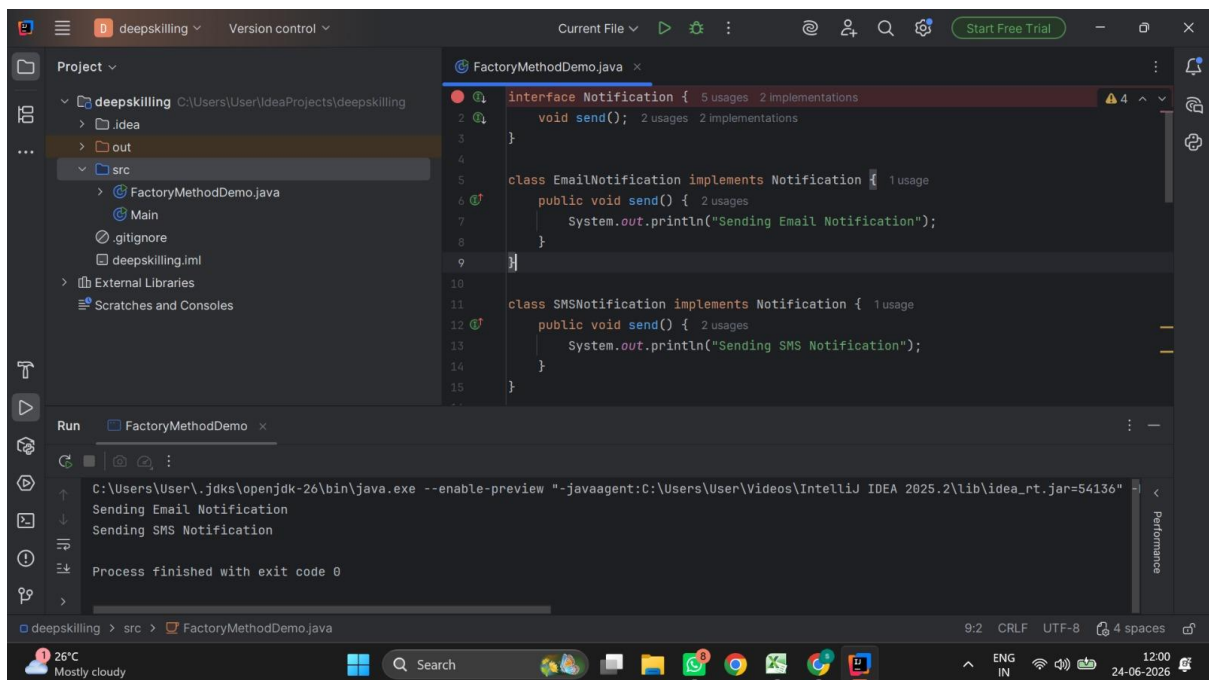
The screenshot shows the IntelliJ IDEA interface with the file `SingletonDemo.java` open. The code implements the Singleton pattern. The `main` method creates two instances of `Singleton` and checks if they are the same object.

```
24 public class SingletonDemo {
25     public static void main(String[] args) {
26         Singleton s1 = Singleton.getInstance();
27         s1.display();
28
29         Singleton s2 = Singleton.getInstance();
30
31         // Check whether both objects are same
32         if (s1 == s2) {
33             System.out.println("Both objects are the same.");
34         }
35     }
36 }
37
```

The Run window shows the output:

```
Singleton Object Created
Singleton Method Called
Both objects are the same.
Process finished with exit code 0
```

Exercise 2: Implementing the Factory Method Pattern



The screenshot shows the IntelliJ IDEA interface with the file `FactoryMethodDemo.java` open. The code implements the Factory Method pattern. It defines an interface `Notification` and two classes, `EmailNotification` and `SMSNotification`, both implementing the `send` method.

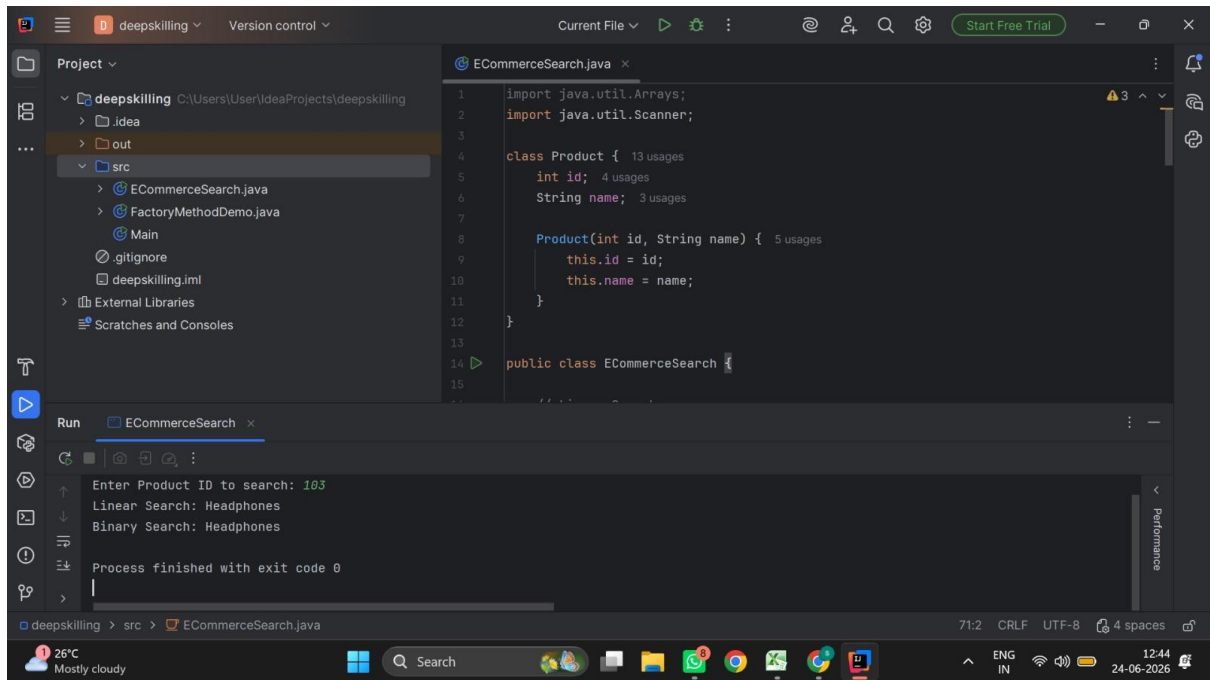
```
1 interface Notification {
2     void send();
3 }
4
5 class EmailNotification implements Notification {
6     public void send() {
7         System.out.println("Sending Email Notification");
8     }
9 }
10
11 class SMSNotification implements Notification {
12     public void send() {
13         System.out.println("Sending SMS Notification");
14     }
15 }
16
```

The Run window shows the output:

```
Sending Email Notification
Sending SMS Notification
Process finished with exit code 0
```

Data structures and Algorithms

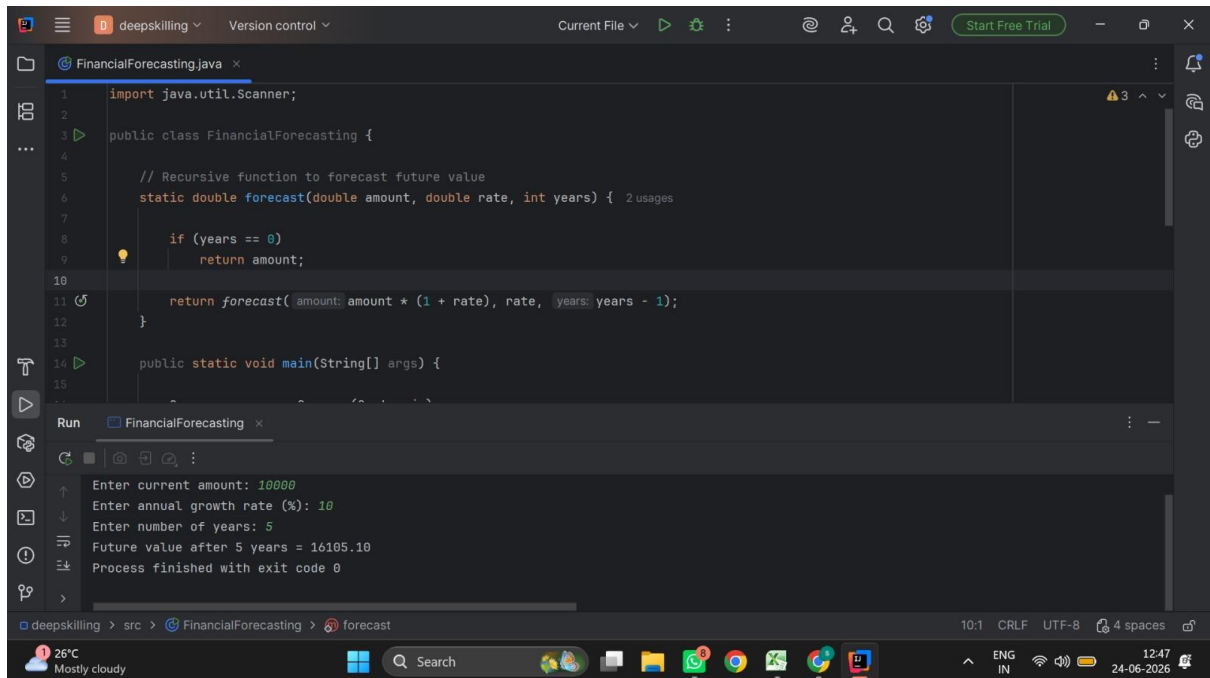
Exercise 2: E-commerce Platform Search Function



The screenshot shows the IntelliJ IDEA IDE with the file `ECommerceSearch.java` open. The code defines a `Product` class with `int id` and `String name` attributes, and a constructor. Below it, the `ECommerceSearch` class is partially visible. The Run window at the bottom shows the execution output:

```
Enter Product ID to search: 103
Linear Search: Headphones
Binary Search: Headphones
Process finished with exit code 0
```

Exercise 7: Financial Forecasting

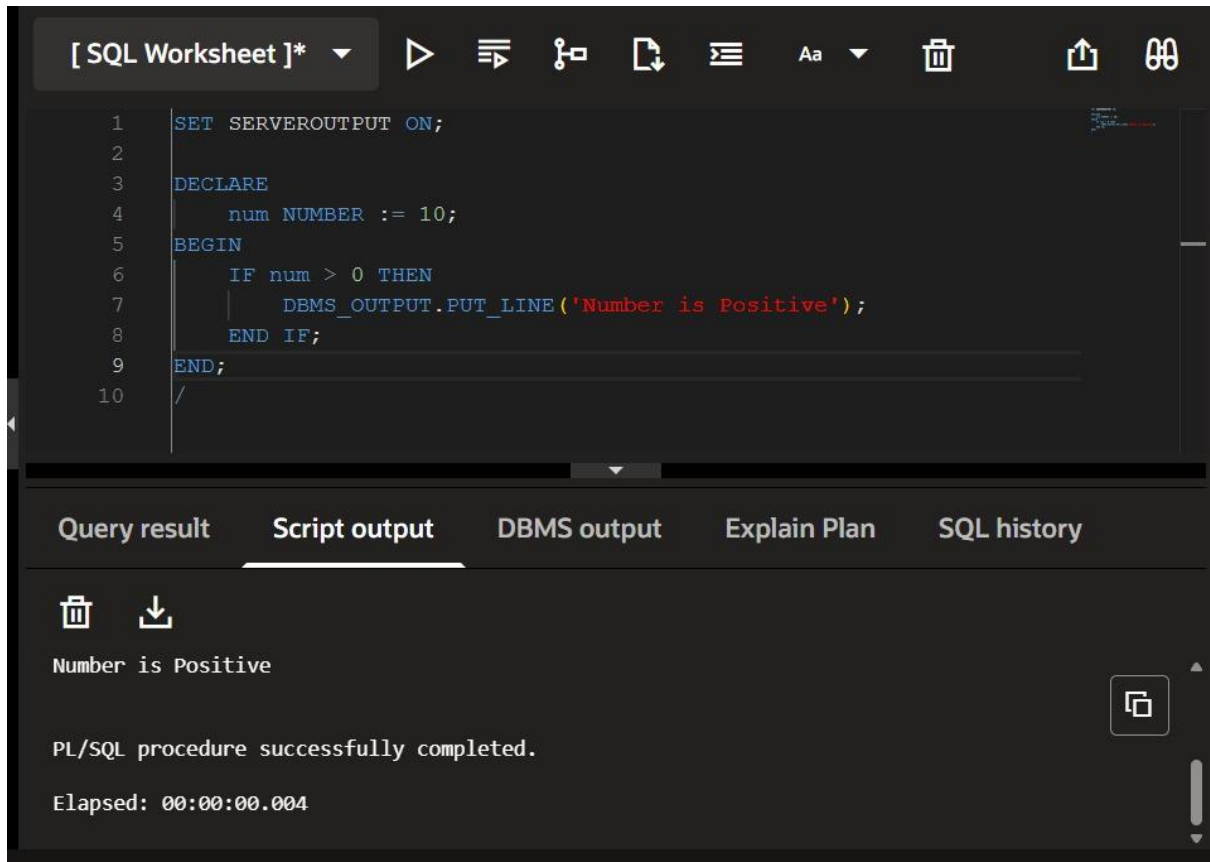


The screenshot shows the IntelliJ IDEA IDE with the file `FinancialForecasting.java` open. The code defines a `FinancialForecasting` class with a recursive `forecast` method and a `main` method. The Run window at the bottom shows the execution output:

```
Enter current amount: 10000
Enter annual growth rate (%): 10
Enter number of years: 5
Future value after 5 years = 16105.10
Process finished with exit code 0
```

PL/SQL programming

Exercise 1: Control Structures



The screenshot shows a SQL Worksheet interface with a dark theme. The top toolbar includes icons for running, saving, and other functions. The main editor contains the following PL/SQL code:

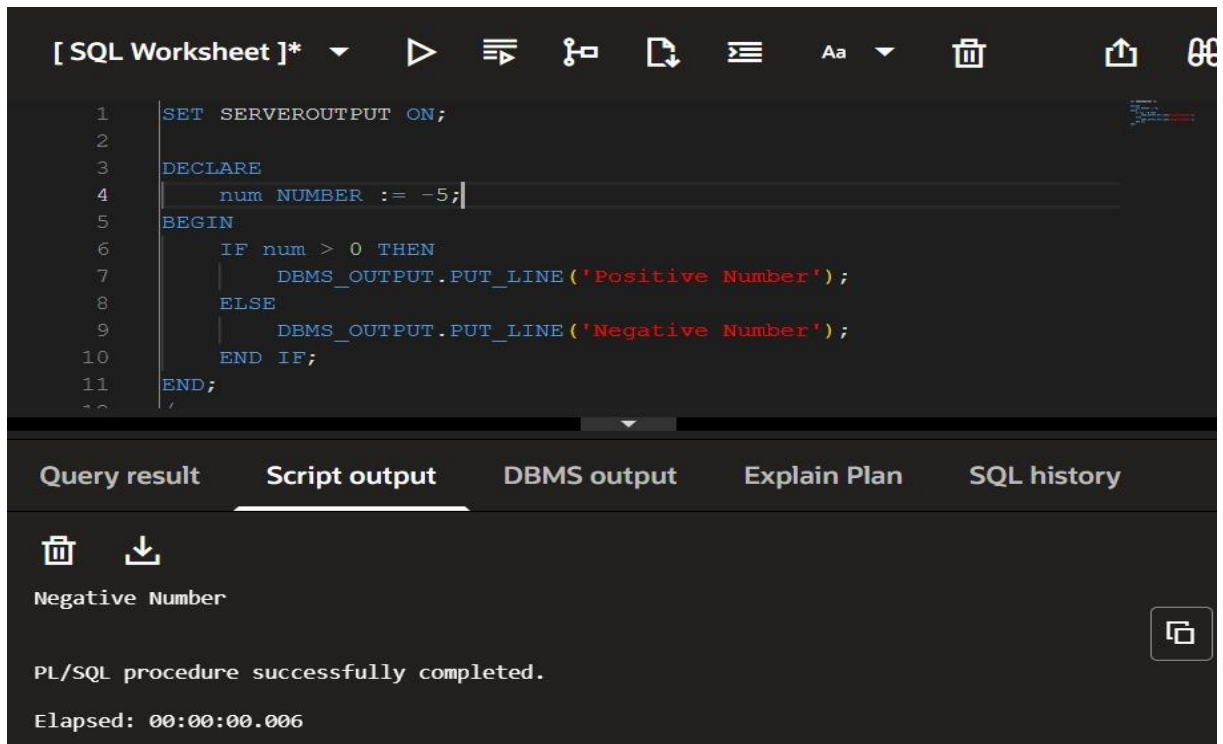
```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     num NUMBER := 10;
5 BEGIN
6     IF num > 0 THEN
7         DBMS_OUTPUT.PUT_LINE('Number is Positive');
8     END IF;
9 END;
10 /
```

Below the editor, the 'Script output' tab is selected, displaying the results of the execution:

```
Number is Positive

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.004
```



The screenshot shows a SQL Worksheet interface with a dark theme. The top toolbar includes icons for running, saving, and other functions. The main editor contains the following PL/SQL code:

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     num NUMBER := -5;
5 BEGIN
6     IF num > 0 THEN
7         DBMS_OUTPUT.PUT_LINE('Positive Number');
8     ELSE
9         DBMS_OUTPUT.PUT_LINE('Negative Number');
10    END IF;
11 END;
12 /
```

Below the editor, the 'Script output' tab is selected, displaying the results of the execution:

```
Negative Number

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.006
```

